The return value from Struct.new(...) is a class object. By assigning it to the constant Person, we can thereafter use Person as if it were any other class.

But say we wanted to change the to_s method of our structure. We could do it by opening up the class and writing the following method:

```
Person = Struct.new(:name, :address, :likes)
class Person
  def to_s
    "#{self.name} lives in #{self.address} and likes #{self.likes}"
  end
end
```

However, we can do this more elegantly (although at the cost of an additional class object) by writing this:

```
class Person < Struct.new(:name, :address, :likes)
  def to_s
    "#{self.name} lives in #{self.address} and likes #{self.likes}"
  end
end

dave = Person.new('Dave', 'Texas')
dave.likes = "Programming Languages"
puts dave
```

produces:

```
Dave lives in Texas and likes Programming Languages
```

This mechanism is useful enough that Struct provides a shortcut by taking a block in which you can define your own methods on the Struct, like this:

```
Person = Struct.new(:name, :address, :likes) do
  def to_s
    "#{self.name} lives in #{self.address} and likes #{self.likes}"
  end
end

dave = Person.new('Dave', 'Texas')
dave.likes = "Programming Languages"
puts dave
```

produces:

```
Dave lives in Texas and likes Programming Languages
```

Using Data for Immutable Structs

Structs are often used to create small classes that represent constrained, encapsulated values with minimal behavior, but where it's useful to be able to treat the value as one thing, like a point with an x and y coordinate. It's also sometimes useful to have those values be *immutable*, meaning that their values cannot be changed. Immutable objects can be easier to reason about and share—they are much easier to manage sharing across threads.

True immutability in Ruby is hard, but you can achieve it with the Data class, new in Ruby 3.2. You use Data.define to create a new class with a particular set of attributes. Once you've created the new class, you can construct instances using either positional or keyword arguments.